1. What are Filters in Java?

Definition

A **Filter** in Java (Servlet API) is an object that is used to **intercept requests and responses** before they reach a servlet or after they leave it.

It acts like a **pre-processing and post-processing layer**.

Why are Filters needed?

Filters are used when you want to apply **common functionality** to multiple resources without repeating code.

Common Uses of Filters

- Authentication & Authorization
- Logging requests
- Data compression
- Input validation
- Encryption / Decryption
- Modifying request/response

How Filter Works

Client → Filter → Servlet → Filter → Client

Filter can:

- Modify request before servlet
- Modify response after servlet

Example

```
import javax.servlet.*;
import java.io.IOException;

public class MyFilter implements Filter {

   public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

      System.out.println("Before Servlet");
```

```
        chain.doFilter(request, response); // pass request आगे

        System.out.println("After Servlet");
    }
}
```

2. Filter Lifecycle

A filter follows a lifecycle managed by the **web container**.

Lifecycle Methods

1. init(FilterConfig config)

- Called **once** when filter is created
- Used for initialization

2. doFilter(ServletRequest, ServletResponse, FilterChain)

- Called for **every request**
- Main processing method
- Must call chain.doFilter() to continue

3. destroy()

- Called once when filter is destroyed
- Used for cleanup

Lifecycle Flow

init() → doFilter() → destroy()

3. Configuring Filters in web.xml

Step 1: Define Filter

```
<filter>
    <filter-name>MyFilter</filter-name>
    <filter-class>com.example.MyFilter</filter-class>
</filter>
```

Step 2: Map Filter to URL

```
<filter-mapping>
   <filter-name>MyFilter</filter-name>
   <url-pattern>/*</url-pattern>
</filter-mapping>
```